

CENTRE FOR SYSTEMS BIOLOGY AND BIOINFORMATICS

University Institute of Engineering and Applied Sciences (U.I.E.A.S.T.)

Panjab University, Chandigarh — 160014

PROJECT REPORT

RAISE Pipeline:

RAISE ~ Research Assessment Intelligence & Semantic Extraction

Stage 1 & 2: Layout-Aware PDF Parsing and Token-Budget Filtering

Submitted by

HARSHDEEP SINGH

M.Sc. Systems Biology and Bioinformatics — 2nd Year

Supervised under

CODATA OT-ViRARE Task Group

Open Tools and Visitation Frameworks for Global Research Assessment Reform

Academic Year 2026–27 | June – July 2026

1. Summary

This project report documents the design, development, testing, and deployment of Stages 1 and 2 of the RAISE (Research Assessment and Institutional Semantification Engine) Pipeline, an open-source, AI-assisted data processing system built under the auspices of the CODATA OT-ViRARE (Open Tools and Visitation Frameworks for Global Research Assessment Reform) Task Group. The broader pipeline converts unstructured university annual reports into validated, machine-readable institutional knowledge graphs suitable for research assessment, policy analysis, and knowledge-graph-based querying.

The work described herein — carried out by Harshdeep Singh as Person B in a four-member collaborative team — focuses on the first two stages of this five-stage pipeline:

- Stage 1: Layout-Aware PDF Parsing — extracting structured text blocks, table rows, font metadata, bounding-box coordinates, and reading order from raw annual-report PDFs using PyMuPDF and pdfplumber.
- Stage 2: Token-Budget Filtering — deterministically removing non-research content (financial audit pages, photo galleries, staff directories, and administrative notices) from the parsed blocks to produce a clean, research-only block JSON file.

The primary deliverable, `target_blocks.json`, was validated in a proven end-to-end integration with Stage 3 (HTML Semantification, owned by Person C): 273 blocks from a real university annual report were passed directly into Person C's `build_report()` module with zero translation required, zero validation issues, and 503 fully-traceable HTML elements generated. The complete module passes 24 automated tests covering all parser behaviours, all four filter categories, CLI commands, and cross-module contract compatibility.

Key Objectives

- Convert raw university annual-report PDFs into structured, layout-aware JSON blocks with full traceability metadata.
- Remove financial, administrative, and noise content before AI extraction, reducing LLM token cost and hallucination risk.
- Produce `target_blocks.json` conforming to the inter-stage contract required by Stage 3.
- Ensure every exclusion is auditable via `filtering_report.json`.

Key Outcomes

Metric	Result
Stage 1 blocks extracted (20-page PDF)	273 blocks with full font, bbox, and table metadata

Metric	Result
Stage 2 filter categories	4 page-level + running header/footer boilerplate stripping
Integration with Stage 3	✓ Zero issues — 273 → 503 HTML elements, all traceable
Automated test cases	24 passing (parser, filter, CLI, handoff compatibility)
Parse time (150-page report)	~3 s (reduced from ~8 s after pdfplumber single-open fix)
Lint & formatting	ruff + black — all checks passed

2. Project Overview

2.1 Project Title

RAISE Pipeline — Stage 1 (Layout-Aware PDF Parsing) and Stage 2 (Token-Budget Filtering)

Module folder: RAISE_PDF_Parsing | Branch: personB-pdf-parsing | Repository: github.com/semanticClimate/RAISE

2.2 Background

University annual reports are the primary public record of an institution's research activity. They contain grants received, publications produced, faculty achievements, patents filed, collaborations established, conference participations, student outcomes, open-science indicators, outreach activities, and national ranking metrics. However, these reports are typically published as large, multi-column PDFs with inconsistent formatting, photo galleries, financial appendices, and administrative sections interleaved with research content. They are not machine-readable in any structured sense.

CODATA (Committee on Data of the International Science Council) established the OT-ViRARE Task Group to address the gap between how research is now conducted — AI-driven, data-intensive, software-mediated, and collaborative — and how it continues to be assessed, which remains anchored in publication-centric and metric-heavy models. A key deliverable of this task group is the RAISE Pipeline: an open, reusable, responsible toolkit for extracting validated research assessment data from institutional reports.

2.3 Business / Institutional Problem

- University annual reports range from 80 to 400 pages and combine research content with financial statements, staff directories, photo galleries, and administrative circulars.
- Feeding a raw annual report directly to a large language model (LLM) causes token overflow, hallucinated metrics, missed sections, and cross-department contamination.
- There is no existing open-source tool that converts such reports into validated, traceable, machine-readable research assessment profiles.

2.4 Project Goals

- Build a production-quality Python module that parses any university annual-report PDF into structured JSON blocks.
- Filter those blocks to retain only research-assessment-relevant content.
- Produce a target_blocks.json file that passes without modification into Person C's HTML Semantification module.
- Ensure every stage is auditable, deterministic, and team-safe.

2.5 Scope

In scope:

- Stage 1: PDF parsing using PyMuPDF (text/font/bbox) and pdfplumber (tables).
- Stage 2: Page-level filtering across four noise categories plus boilerplate stripping.
- A Typer-based CLI with three commands: parse, filter, run.
- 24-test pytest suite including cross-module integration tests.
- A Google Colab notebook for one-click execution.
- JSON Schema files for both stage outputs (report_blocks.json and target_blocks.json).

Out of scope:

- HTML Semantification (Stage 3 — Person C).
- AI Structured Extraction (Stage 4 — Person D).
- Ontological Validation with ORCID/ROR/Wikidata (Stage 5 — Person D).
- Dossier schema definition (Person A).
- OCR for scanned PDFs (noted as future enhancement).

2.6 Assumptions

- [ASSUMPTION] Annual reports are born-digital PDFs with an embedded text layer; OCR is not required for the current phase.
- [ASSUMPTION] The team uses Python 3.10+ and has access to GitHub for branch-based collaboration.
- [ASSUMPTION] Cloud API access for LLM extraction (Stage 4) is available via an institutional or personal key; this module does not require it.

2.7 Constraints

- Two-week sprint for the initial working demo (14 days).
- Free-tier Google Colab runtime (2 vCPUs, ephemeral storage).
- No GPU required: all stages in this module are CPU and I/O operations.
- All code must be Ruff and Black clean to comply with team coding standards.

3. Requirements

3.1 Functional Requirements

ID	Requirement	Priority	Stage
FR-01	Accept a raw PDF as input and extract text blocks with page number, block ID, bbox, text content, font size, font name, bold/italic flags, and reading order.	Must Have	Stage 1
FR-02	Detect and extract well-formed tables as row arrays using pdfplumber's ruling-line table detector.	Must Have	Stage 1
FR-03	Assign a stable, deterministic block_id to every block (format: p{page}-b{index:03d}) that does not change between runs on the same PDF.	Must Have	Stage 1
FR-04	Detect two-column page layouts and produce reading order that processes the left column before the right column.	Should Have	Stage 1
FR-05	Assign a block_type_guess (heading / paragraph / list_item / table / page_number) using font-size and	Must Have	Stage 1

ID	Requirement	Priority	Stage
	bold heuristics.		
FR-06	Propagate a section_hint from the most recent heading block forward so downstream stages know which research section a block belongs to.	Must Have	Stage 1
FR-07	Exclude financial/audit pages, photo gallery pages, staff directory pages, and admin notice pages from target_blocks.json.	Must Have	Stage 2
FR-08	Detect and strip repeating running headers and footers (short text blocks in the page margin that repeat across 3+ pages).	Must Have	Stage 2
FR-09	Log every excluded page with its page number, exclusion category, human-readable reason, and matched signals in filtering_report.json.	Must Have	Stage 2
FR-10	Expose parse, filter, and run commands via a Typer CLI with standard --input/--output flags.	Must Have	Both

3.2 Non-Functional Requirements

ID	Requirement	Target
NFR-01	Performance — parse time for a 150-page annual report	≤ 5 seconds on 2 vCPUs
NFR-02	Reproducibility — running the parser twice on the same PDF must produce identical block IDs and reading order	100% deterministic
NFR-03	Contract compliance — target_blocks.json must validate against the JSON Schema without errors	0 schema violations
NFR-04	Code quality — Ruff and Black compliance with no warnings	All checks passed
NFR-05	Test coverage — all public functions must have at least one test case	24 tests, 0 failures
NFR-06	Filter conservatism — never exclude a page that contains research content unless it clearly matches a noise category with score ≥ 2	0 false positive exclusions on demo PDFs
NFR-07	Auditability — every block must carry source_file and block_id for end-to-end traceability	100% blocks traceable

4. Project Architecture

4.1 Overall Five-Stage Pipeline

The RAISE Pipeline is composed of five sequentially linked stages. Each stage reads from a fixed input path, produces a fixed output path, and exposes a well-defined JSON schema that acts as a contract for the downstream stage. This design prevents team members from overwriting each other's work and makes each stage independently testable and replaceable.

Stage	Name	Owner	Input	Output
1	Layout-Aware PDF Parsing	Person B (Harshdeep)	/data/01_raw/report.pdf	report_blocks.json
2	Token-Budget Filtering	Person B (Harshdeep)	report_blocks.json	target_blocks.json
3	HTML Semantification	Person C	target_blocks.json	report.html
4	AI Structured Extraction	Person D	report.html + dossier_schema.json	raw_metrics.json
5	Ontological Validation	Person D	raw_metrics.json	institutional_profile.json

4.2 Person B Module Components

The RAISE_PDF_Parsing module (src/raise_pdf_parsing/) contains the following source files:

File	Responsibility
models.py	Pydantic data models: Block, PageStats, ReportBlocksDocument, TargetBlocksDocument, FilteringMetadata, ExclusionRecord.
utils.py	Shared utilities: slugify, word_count, text_confidence (extraction-integrity score), normalize_whitespace.
classify.py	Heuristic block-type classifier (heading/paragraph/list_item), section-keyword vocabulary aligned with Stage 4

File	Responsibility
	extraction categories, bullet and page-number detectors.
parser.py	Stage 1 implementation: text extraction (PyMuPDF), table extraction (pdfplumber opened once per document), column detection, reading-order assignment, block assembly.
filter.py	Stage 2 implementation: page-level noise classification (four categories), running header/footer detection via page-number-aware key normalisation, block retention loop.
main.py	Orchestration entry points: run_parse(), run_filter(), run_pipeline(). Owns all file I/O.
cli.py	Typical CLI with parse, filter, and run commands; Rich console output with summary tables.

4.3 Technologies Used

Technology	Version	Purpose
Python	3.10+	Primary language for all pipeline logic.
PyMuPDF (fitz)	1.24+	Text extraction with font, bbox, and bold/italic metadata. Opened once per page inside the fitz.open() context manager.
pdfplumber	0.11+	Table detection using ruling-line and text-alignment strategy. Opened once for the whole document to avoid per-page overhead.
Pydantic v2	2.7+	Strict data models for all inter-stage JSON contracts with automatic validation.

Technology	Version	Purpose
Typer + Rich	0.12+	CLI framework and terminal output (summary tables, colour-coded results).
pytest	8.2+	Test runner for 24 unit, integration, and CLI tests.
Ruff + Black	Latest	Linting and code formatting — consistent with team standards.
JSON Schema (Draft 2020-12)	–	Schema files for both report_blocks.json and target_blocks.json (schemas/ folder).
Google Colab	–	Notebook environment for demo and team usage.
GitHub	–	Version control; branch personB-pdf-parsing; no team member edits another's folder without PR review.

4.4 Repository Structure

The module lives in the RAISE monorepo under the folder `RAISE_PDF_Parsing/` with the following layout:

```

RAISE_PDF_Parsing/ |— data/                                # Stage I/O data (gitignored
except .gitkeep) |   |— 01_raw/                            # raw input PDFs |   |— 02_parsed/
# report_blocks.json (Stage 1 output) |   |— 03_filtered/    #
target_blocks.json + filtering_report.json |— schemas/      # JSON
Schema files for both outputs |— src/raise_pdf_parsing/ # Python package
(models, parser, filter, CLI) |— tests/                    # pytest suite (24
tests) |   |— fixtures/                                    # synthetic PDF generator (make_sample_pdf.py)
|— notebooks/      # Colab notebook |— docs/              #
HANDOFF.md – contract doc for Person C |— outputs/        # demo output
files

```

5. Methodology

5.1 Development Approach

The project followed an Agile sprint model with a defined two-week execution plan as specified in the RAISE Pipeline project deck (Slide 17). Short daily check-ins were used to synchronise with the other team members (Persons A, C, and D). GitHub pull requests enforced the governance rule that no team member edits another member's folder without review.

5.2 Project Phases

Phase	Days	Key Activities	Deliverable
Setup	Day 1–2	Create repo, freeze folder structure, add sample PDFs, begin Pydantic models, study Person C's existing Colab notebook to understand Stage 3 input contract.	models.py, schemas/, pyproject.toml
Stage 1 Prototype	Day 3–5	Build parser.py using PyMuPDF for text and pdfplumber for tables. Implement reading-order and two-column detection. First report_blocks.json produced.	parser.py, report_blocks.json
Stage 2 Filtering	Day 6–8	Build filter.py with four noise categories and boilerplate stripping. Discover and fix pdfplumber per-page open bug. Write classify.py heuristics.	filter.py, filtering_report.json
Testing & CLI	Day 9–12	Write 24 pytest tests including the synthetic PDF	tests/, cli.py, 24 tests passing

Phase	Days	Key Activities	Deliverable
		generator. Build Typer CLI. Run ruff and black. Test against real uploaded PDF.	
Integration & Notebook	Day 13–14	Validate target_blocks.json against Person C's build_report(). Write Colab notebook. Prepare handoff documentation (docs/HANDOFF.md).	Verified integration, Colab notebook

6. Implementation

6.1 Stage 1 — Layout-Aware PDF Parsing

Stage 1 is implemented in `parser.py` and is the most technically complex module. It uses two libraries in combination because each is better at a different task: PyMuPDF provides high-fidelity text extraction with full font and coordinate metadata but poor table detection; pdfplumber provides accurate table detection via ruling-line analysis but limited font metadata.

Text Block Extraction

For each page, `page.get_text("dict")` is called via PyMuPDF, which returns a hierarchical structure of blocks, lines, and spans. The dominant font attributes (font size, font name, bold, italic) are computed by weighting each span by its character count, so a two-character superscript footnote marker does not override the paragraph's real font. Text is normalised (whitespace collapsed) before being stored.

Table Extraction

pdfplumber's `find_tables()` method uses an intersection-based ruling-line detector to identify table regions. It is called once per document (not once per page — a critical performance fix described in Section 7). Each detected table yields a list of rows, each row being a list of cell strings. These are stored in the block's `rows` field. Text blocks that overlap more than 60% with a detected table are suppressed to prevent duplicate content.

Two-Column Detection

A simple but effective column detector examines the horizontal centre point of each block. If the page has at least four narrow blocks (width < 55% of page width) with at least two centred in the left half and two in the right half, the page is classified as multi-column. Blocks are then sorted left-column first, right-column second, with each sub-column sorted top-to-bottom within it.

Classification and Section Hints

Block type classification uses three combined signals: font size relative to the page's median body font size, bold flag, and word count. A heading score is computed; if it exceeds 0.5, the block is classified as heading. The `section_hint` is updated whenever a heading block matches a keyword from the `SECTION_KEYWORDS` vocabulary (Grants, Publications, Patents, Faculty, Collaborations, Conferences, Student Outcomes, Open Science, Outreach, Ranking, Department, Awards) and carried forward to all subsequent blocks until the next recognised heading.

6.2 Stage 2 — Token-Budget Filtering

Stage 2 is implemented in `filter.py`. It reads the `ReportBlocksDocument` produced by Stage 1 and applies a page-level exclusion step followed by a block-level boilerplate removal step.

Page-Level Noise Classification

Each page is evaluated against four ordered categories:

- Financial/Audit: keyword match against a vocabulary of 14 financial-statement phrases (balance sheet, auditor's report, trial balance, depreciation schedule, etc.). A page is excluded when at least two keywords match.
- Photo Gallery: structural — excluded when image area covers more than 30% of the page AND word count is below 60 AND at least two images are embedded. No keyword matching is needed.
- Staff Directory: regular-expression match for the pattern Name (Dr./Prof./Mr./Ms.) followed by a role title (Professor, Associate Professor, Dean, HOD, Registrar, etc.). Four or more such rows on a single page triggers exclusion.
- Admin Notice: keyword match against a vocabulary of 11 administrative-notice phrases (circular no., office order, minutes of the meeting, tender notice, attendance register, etc.).

Boilerplate Detection and Removal

Running headers and footers are detected by finding short text blocks (fewer than 14 words) that sit in the top or bottom 8% of the page and whose normalised key appears on three or more pages. The key normalisation is page-number-aware: a trailing or leading numeric token is stripped only if it exactly matches that block's own page number, preventing legitimate per-page content such as stage kicker labels (STAGE 1, STAGE 2) from being misidentified as boilerplate.

6.3 Workflow

The standard execution workflow is:

- Deposit the raw PDF at data/01_raw/report.pdf.
- Run `python -m raise_pdf_parsing run` (or the Google Colab notebook).
- Stage 1 output written to data/02_parsed/report_blocks.json.
- Stage 2 output written to data/03_filtered/target_blocks.json and data/03_filtered/filtering_report.json.
- target_blocks.json is uploaded to Person C's RAISE_HTML5_Semantification_Colab.ipynb.

6.4 Inter-Stage API Contract

The inter-stage contract is defined by the JSON Schema at schemas/target_blocks.schema.json and documented in docs/HANDOFF.md. The key fields that Stage 3 depends on are: block_id (stable traceability), page_number, text, bbox, font_size, is_bold, reading_order, block_type_guess, section_hint, rows (for tables), and confidence. These field names were chosen to exactly match the FIELD_ALIASES already present in Person C's raise_html5_semantification.models module, so no alias translation is required on either side of the handoff.

7. Challenges

Challenge	Category	Solution Implemented
pdfplumber opened the PDF once per page inside the parsing loop, causing 150 file-open cycles on a large report and adding ~5 s of overhead.	Performance	Refactored to <code>_extract_all_table_blocks()</code> which opens pdfplumber once for the whole document and returns a dict keyed by page number. Parse time dropped from ~8 s to ~3 s on a 150-page report.
Overly aggressive digit-normalisation in the boilerplate key function collapsed legitimate per-page labels (STAGE 1, STAGE 2) into false boilerplate, silently removing them.	Correctness	Replaced blanket digit-collapse with page-number-aware normalisation: a numeric token is stripped only when it matches that block's own <code>page_number</code> , never as a generic pattern.
Two-column page detection using a half-page boundary with a fixed gap threshold was too strict: blocks with normal column gutters were classified as full-width, making the page appear single-column.	Layout	Replaced with a centre-point heuristic: blocks narrower than 55% of the page width are classified as narrow; if at least four narrow blocks exist with ≥ 2 on each side of the midpoint, the page is treated as two-column.
Table text was appearing both as table rows (from pdfplumber) and as duplicate paragraph blocks (from PyMuPDF), creating redundant content for Stage 3.	Data quality	Implemented <code>_merge_text_and_tables()</code> which suppresses any text block that overlaps more than 60% with a detected table's bounding box, keeping only the structured table representation.
Bold attribute weighting: a short superscript or footnote marker in one span was overriding the dominant font of the	Parsing	Implemented <code>_dominant_span_attrs()</code> which weights each span by character count, so a 2-character footnote cannot

Challenge	Category	Solution Implemented
whole block.		change the classification of a 200-character paragraph.
Filtering rules needed to be conservative (avoid false-positive exclusions) without also being so permissive that audit pages slipped through to the LLM stage.	Policy	Set KEYWORD_SCORE_THRESHOLD = 2: a page must match at least two independent keywords before being excluded. Ambiguous pages are always retained and passed downstream, consistent with the project's governance principle.

8. Testing

8.1 Test Strategy

Testing was performed using pytest with a session-scoped fixture that builds a fully synthetic seven-page annual report PDF (tests/fixtures/make_sample_pdf.py) in memory. The synthetic PDF was engineered to exercise every behaviour the module is expected to handle: a research-narrative page, a ruled table page, a financial-audit page, a photo-gallery page, a staff-directory page, an admin-notice page, and a two-column research page — all wrapped in a running header and footer repeated on every page.

The test suite is divided into four files: test_parser.py (Stage 1 unit tests), test_filter.py (Stage 2 unit tests), test_cli.py (end-to-end CLI tests), and test_handoff_compatibility.py (integration tests that install Person C's raise_html5_semantification package and call build_report() directly against this module's output).

8.2 Test Cases Summary

Test File	Test Name	Validates
test_parser.py	test_parsing_every_page	7 pages parsed, blocks > 0
test_parser.py	test_blocks_have_stable_deterministic_ids	IDs identical on two runs
test_parser.py	test_reading_order_is_monotonic	Strictly increasing, no duplicates
test_parser.py	test_heading_detected_with_bold_large_font	block_type_guess == heading, section_hint == Research
test_parser.py	test_bullet_list_items_classified	Lines starting with - classified as list_item
test_parser.py	test_table_extracted_with_rows	Table has real rows; header row matches expected
test_parser.py	test_table_rows_not_duplicated_as_paragraphs	No standalone paragraph block contains table text
test_parser.py	test_two_column_page_reads_left_before_right	Left column blocks appear before right column blocks in reading_order
test_parser.py	test_page_stats_capture_images_and_words	Photo gallery page: image_count==4, image_area_ratio>0.3
test_parser.py	test_confidence_in_valid_r	All confidence values in

Test File	Test Name	Validates
	ange	[0.0, 1.0]
test_filter.py	test_financial_audit_page_excluded	Page 3 excluded as financial_audit
test_filter.py	test_photo_gallery_page_excluded	Page 4 excluded as photo_gallery
test_filter.py	test_directory_page_excluded	Page 5 excluded as directory
test_filter.py	test_admin_notice_page_excluded	Page 6 excluded as admin_notice
test_filter.py	test_research_pages_retained	retained_pages == [1, 2, 7]
test_filter.py	test_running_header_and_footer_removed	University name and confidentiality notice absent from output
test_filter.py	test_research_content_survives_filtering	RESEARCH ACTIVITIES, PUBLICATIONS, FACULTY ACHIEVEMENTS present
test_filter.py	test_table_block_survives_with_rows_intact	Table block in output has 4 rows
test_filter.py	test_block_count_accounting_is_consistent	output_block_count == len(target.blocks)
test_filter.py	test_every_exclusion_has_a_reason	All ExclusionRecord objects have reason and matched_signals
test_cli.py	test_run_command_writes_all_three_outputs	All three files created; block count consistent
test_cli.py	test_parse_then_filter_commands_separately	parse then filter independently produces valid output
test_handoff.py	test_target_blocks_json_loads_into_stage3_without_translation	load_blocks() succeeds; no generated IDs needed
test_handoff.py	test_target_blocks_json_produces_valid_traceable_html	validation.ok == True; 0 issues; Balance Sheet and Circular No absent from HTML

8.3 Test Results

All 24 tests pass. The test suite runs in approximately 3.8 seconds on a standard laptop (3 iterations × 20-page PDF). Ruff reports zero lint issues and Black reports all 17 source files unchanged.

9. Project Timeline

Phase	Days	Deliverables	Status
Project Setup	Day 1–2	Repo created, branch personB-pdf-parsing, models.py, pyproject.toml, schema stubs.	✓ Complete
Stage 1 Prototype	Day 3–5	parser.py with PyMuPDF text extraction + pdfplumber tables. First report_blocks.json on sample PDF.	✓ Complete
Stage 2 Filtering	Day 6–8	filter.py with four noise categories, boilerplate detection, filtering_report.json.	✓ Complete
Bug Fixes	Day 9	pdfplumber single-open fix, column-detection rework, digit-normalisation fix.	✓ Complete
Testing & CLI	Day 10–11	24 pytest tests, Typer CLI, Ruff + Black compliance, synthetic PDF fixture.	✓ Complete
Integration Validation	Day 12	Cross-module test with Person C's build_report(). 273 → 503 elements, 0 issues.	✓ Complete
Documentation & Colab	Day 13–14	README.md, docs/HANDOFF.md, JSON Schemas, Colab notebook,	✓ Complete

Phase	Days	Deliverables	Status
		project report.	

10. Results

10.1 Performance Metrics

KPI	Target	Achieved	Status
Parse time (150-page PDF)	≤ 5 s	~3 s	✓ Met
Schema validation errors on output	0	0	✓ Met
Test failures	0 / 24	0 / 24	✓ Met
Ruff lint issues	0	0	✓ Met
Stage 3 integration validation issues	0	0	✓ Met
Traceable HTML elements from real PDF	All blocks traceable	503 / 503 (100%)	✓ Met
False-positive page exclusions (demo PDFs)	0	0	✓ Met

10.2 Key Achievements

- Built and tested a fully functional Stage 1 + Stage 2 Python module in 14 days.
- Identified and fixed a critical performance bug (pdfplumber opened once per page instead of once per document) that reduced parse time by 62%.
- Proved end-to-end integration with Stage 3: 273 blocks produced by this module were accepted by Person C's build_report() without any field-name translation, producing 503 fully traceable HTML elements with zero validation issues.
- Established and documented the JSON inter-stage contract (schemas/docs/HANDOFF.md) that defines the boundary between Person B's and Person C's modules.
- Delivered a Google Colab notebook with a built-in smoke test that verifies all four filter categories before processing the user's actual PDF.

11. Risk Analysis

Risk	Impact	Probability	Level	Mitigation
Annual report is a scanned PDF with no text layer (image-only)	High	Medium	HIGH	Future: add Tesseract/Easy OCR fallback. Current: parser returns empty blocks with low confidence for affected pages.
LLM hallucination in Stage 4 due to badly structured Stage 3 HTML	High	Low	MED	Mitigated by Stage 2 filtering (clean input to Stage 3) and Stage 5 validation with human review for low-confidence entries.
Keyword lists for filter categories tuned for English-language Indian university reports — may miss international report structures	Medium	Medium	MED	Keyword vocabularies are configurable per-institution. Person A reviews and extends them as part of dossier definition.
Schema mismatch between Stage 2 output and Stage 3 input after independent updates	High	Low	MED	test_handoff_compatibility.py runs on every PR and catches any mismatch immediately.
Google Colab	Low	High	MED	Install cell re-

Risk	Impact	Probability	Level	Mitigation
runtime reset losing installed packages mid-session				runs in under 90 s. Notebook includes runtime-reset detection instructions in markdown.
Team member pushes to another member's folder without PR review	Medium	Low	LOW	Folder ownership is documented in README and enforced by branch protection rules; .gitignore prevents accidental commits of data files.

12. Budget (Estimated)

[ASSUMPTION] The project was developed as an academic open-source contribution. No direct financial expenditure was incurred. The following table estimates the equivalent monetary value of tools and infrastructure used.

Resource	Provider	Cost	Notes
Python, PyMuPDF, pdfplumber, pytest, Ruff, Black, Typer	Open Source	₹0	All permissively licensed (MIT / Apache 2.0)
Google Colab (CPU runtime)	Google	₹0	Free tier sufficient for PDF parsing tasks
GitHub (private repo)	GitHub	₹0	Free for academic accounts under GitHub Education
LLM API (Stage 4 — Person D, not this module)	Gemini / Claude API	~₹800/month est.	[ASSUMPTION] For Stage 4 use; not required for Stages 1–2
Developer time (Person B, 14 days × 6 h/day)	N/A (academic)	~₹21,000 equiv.	[ASSUMPTION] At ₹250/h market rate for junior Python developer
TOTAL (direct costs)	–	₹0	Open-source tools; academic project

13. Lessons Learned

13.1 What Worked Well

- Defining the inter-stage JSON Schema before writing any parsing code meant the contract between Stage 2 and Stage 3 was clear from day one, and the integration test simply verified what had already been agreed.
- Using Pydantic v2 for all data models caught type errors immediately (e.g., a None being assigned to a float field) rather than producing silent failures downstream.
- Writing the synthetic PDF generator (make_sample_pdf.py) early meant the test suite could exercise all four filter categories without depending on a real university report that could not be committed to the repository.
- The conservative filter threshold (KEYWORD_SCORE_THRESHOLD = 2) prevented false-positive exclusions on both the synthetic and real test PDFs.
- Reading Person C's existing Colab notebook before writing a single line of code revealed exactly what field names Stage 3 expected, allowing the models.py field names to be aligned from the start.

13.2 What Could Be Improved

- The two-column detector works for simple left/right layouts but does not handle three-column or complex magazine-style layouts. A more robust reading-order algorithm (e.g., using vertical strip clustering) would improve accuracy on non-standard annual reports.
- The financial-audit keyword list is tuned for Indian university report conventions. A configurable, institution-specific keyword file (specified via a YAML config) would make the module more portable across different countries' reporting standards.
- pdfplumber's table detection occasionally fails on tables without visible ruling lines (text-only alignment tables). GROBID integration or a machine-learning-based table detector would improve recall on such cases.

13.3 Recommendations for Future Teams

- Always write the inter-stage contract schema and a minimal handoff test before implementing the stage itself. This prevents integration failures discovered only at the end of the sprint.
- Never open a file inside a loop when the file can be opened once outside the loop. The pdfplumber per-page open bug was not obvious from the code but had a 62% performance impact.
- Use a session-scoped synthetic fixture instead of committing real PDFs to the test suite. This keeps the repository clean and the tests fully self-contained.

14. Future Enhancements

Enhancement	Priority	Description
OCR Fallback for Scanned PDFs	High	Integrate Tesseract or EasyOCR as a fallback when PyMuPDF detects an image-only page (no text layer). Required for many older university annual reports.
GROBID Integration	Medium	Use GROBID's bibliography-extraction model for reports that include reference-list style sections, improving recall on publication and citation blocks.
Three-Column Layout Support	Medium	Extend the column detector to handle three-column magazine layouts using vertical strip clustering (e.g., via a k-means approach on block x-centres).
Configurable Filter Rules via YAML	Medium	Externalise keyword vocabularies and thresholds into a per-institution YAML config file so the same codebase can handle reports from universities in multiple countries.
Parallel Page Processing	Low	Parallelise the PyMuPDF page loop using <code>concurrent.futures.ThreadPoolExecutor</code> to further reduce parse time on very large reports (200+ pages).
Dashboard Integration	Low	Export a summary of filtering decisions as a lightweight HTML dashboard so

Enhancement	Priority	Description
		administrators can quickly review which pages were excluded and why, without opening JSON files.
Multi-Language Support	Low	Extend keyword vocabularies to cover Hindi, Tamil, and other Indian-language content that appears in bilingual annual reports.

15. Conclusion

This project demonstrates that deterministic, layout-aware PDF pre-processing is both feasible and essential as the foundation of an AI-assisted research assessment pipeline. By extracting structured blocks with full provenance metadata in Stage 1 and removing noise content in Stage 2, the module ensures that the large language model in Stage 4 never sees financial appendices, photo galleries, or staff directories — only clean, section-labelled research content constrained by a dossier schema.

The work contributes directly to CODATA OT-ViRARE's mission of building open, responsible tools for global research assessment reform. The module is production-quality, fully tested, lint-clean, and has been validated in a real end-to-end integration with Stage 3. The `target_blocks.json` file produced by this module can be handed directly to Person C's `RAISE_HTML5_Semantification` Colab notebook with no translation or reformatting, meeting the project's strict inter-stage contract requirements.

The two-week sprint timeline was met in full. The key technical contribution beyond the stated requirements was identifying and fixing a performance regression in the pdfplumber integration (per-page file-open) that would have made the module impractical on large reports, and designing a boilerplate detection algorithm precise enough to strip running headers and footers without accidentally removing legitimate per-page content.

As an M.Sc. student in Systems Biology and Bioinformatics at Panjab University, this project provided practical experience in applied AI systems design, data engineering, test-driven development, and open-science infrastructure — areas of growing importance at the intersection of computational biology and research data management.

16. References

- CODATA. (2025). Open Tools and Visitation Frameworks for Global Research Assessment Reform (OT-ViRARE) Task Group. International Science Council. <https://codata.org/>
- PyMuPDF Documentation. (2024). PyMuPDF 1.24 — Python Bindings for MuPDF 1.24. <https://pymupdf.readthedocs.io/>
- pdfplumber Documentation. (2024). pdfplumber: Plumb a PDF for detailed information. <https://github.com/jsvine/pdfplumber>
- Pydantic. (2024). Pydantic v2 — Data Validation using Python Type Hints. <https://docs.pydantic.dev/>
- Typer. (2024). Typer — Build great CLIs with Python, easy to code and use. <https://typer.tiangolo.com/>
- RAISE Pipeline Project Vision Deck. (2025). Research Assessment Intelligence & Semantic Extraction Pipeline: From Raw Annual Reports to Validated Institutional Knowledge Graphs. CODATA OT-ViRARE.
- Centre for Systems Biology and Bioinformatics. (2024). M.Sc. Systems Biology and Bioinformatics Programme. Panjab University, Chandigarh. <https://systemsbiology.puchd.ac.in/>
- San Francisco Declaration on Research Assessment (DORA). (2012). <https://sfdora.org/>
- Research Data Alliance and CODATA. (2022). FAIR Data Maturity Model and Case Studies Framework. <https://codata.org/>
- JSON Schema. (2020). JSON Schema: A Vocabulary for Structural Validation of JSON — Draft 2020-12. <https://json-schema.org/>